

Excel for Analyzing Inefficiency and Preference
Manipulation in Two-Sided Matching:
A User's Manual (Concise Edition)*

March 24, 2026

*This manual was prepared by Shuya Abe (Faculty of Management and Information Sciences, Fukui University of Technology) and Naoki Watanabe (Graduate School of Business Administration, Keio University). Last updated on **September 11** in 2026.

Contents

1	Introduction	3
1.1	Overview	3
1.2	Precautions	5
2	Configuration	6
2.1	man sheet	6
2.2	group sheet	7
2.3	conf sheet	7
2.4	result sheet	8
2.5	analyze sheet	10
2.6	better_choices sheet	11
2.7	identical_choices sheet	12
3	Operation	13
3.1	Executing Matching Algorithms: Basic Function	13
3.2	Welfare Evaluation: Inefficient Matching	13
3.3	Preference Manipulation: Strategic Aspects	14
4	Troubleshooting	15
4.1	Developer Ribbon is Not Displayed	15
4.2	Error Message is Displayed	15
4.3	Results are Incorrect (without Error Message)	16

1 Introduction

This note is a manual that explains how to use *Excel for Analyses of Two-Sided Matching* (`searching_choices_in_the_matching.xlsm`). The main part of the program is designed to run the deferred acceptance (DA) and immediate acceptance (IA or Boston) algorithms, featuring the random generation of preferences and priority orders and a heuristic evaluation method of inefficiency of a matching, which is the revised version of *Excel for Two-Sided Matching Ver. 3.2* (`two-sided-matching.xlsm`). We add two functions to the previous one. One is for searching better choices **for each worker** against the preferences stated by the others under the Boston algorithm, and another one is for searching **other choices for each worker** that **assign him or her to the same or better divisions against the preferences stated by the others** under the DA algorithm.

1.1 Overview

The program targets “Many-to-One Matching,” where workers are assigned to one of groups each having its capacity. See the User’s Manual of *Excel for Two-Sided Matching* for the illustration and explanation about the two sided matching, which is available at a GitHub repository. In the repository, click on the green button “Code” to open a drop-down list. Then, choose “Download Zip.” This is the simplest way to download the file. Below is the URL.

<https://github.com/shuya-abe/excel-for-two-sided-matching>

Figure 1 shows the editor screen of the VBA source code which forms the core of this program. As is seen in the screen-shot, notes for the codes are put in Japanese, and that part is displayed in green. Table 1 shows the development environment for this program. The screen-shots attached in this manual were taken in this environment. This program operates in any Microsoft Office Excel environments capable of running VBA (Visual Basic for Applications), but we recommend the operating environment listed in Table 2.

Table 1: Execution Environment Used for This Manual

OS	Windows 11
Excel	Microsoft Office 365 / Excel 2019 or later
CPU	Intel(R) Core(TM) i7-12700F
Memory	32 GB
Disk	2 TB

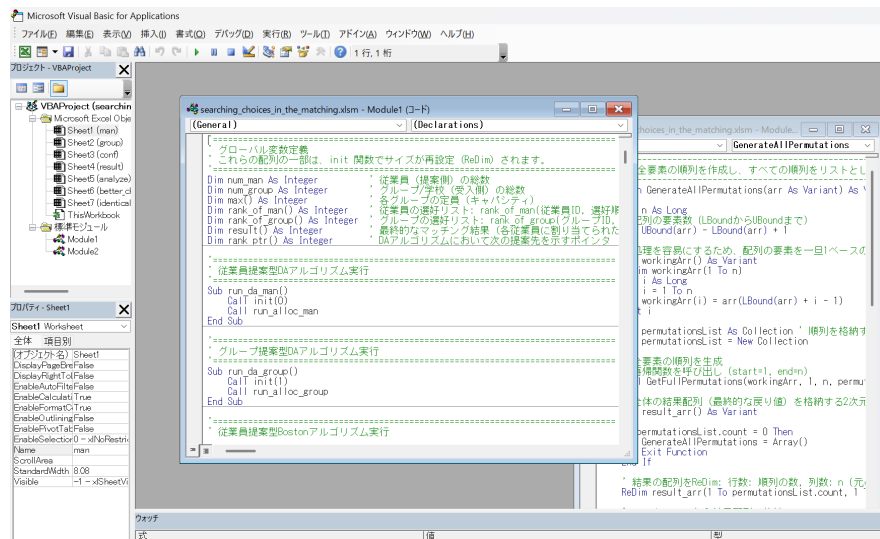


Figure 1: VBA Source Codes in the Editor Screen

Table 2: Recommended Operating Environment

OS	Windows 10 or later, Mac OS X 10.15 or later
Excel	Microsoft Office 2016 or later
CPU	No specific requirements (as long as Excel runs)
Memory	4 GB or more
Disk	No specific requirements

1.2 Precautions

Excel for Analyses of Two-Sided Matching consists of seven Excel sheets (man, group, conf, result, analyze, better_choices, and identical_choices) and one VBA program. Read appropriate textbooks for reading and writing VBA codes.

Two-sided matching is an assignment problem in which, for example, workers are assigned to divisions in a company:

1. Each worker has a preference over the divisions to which he or she is to be assigned, and each division has a priority order regarding the acceptance of those workers. The preferences and priority orders are represented by ranking without ties.
2. Each division has its capacity (or quota), the maximal number of workers it can accept.

The following points should be noted when using this program:

- **Data Input:** When entering initial settings in the VBA program, or workers' preferences, divisions' priority orders, and capacity in each sheet, always use **half-width alphanumeric characters**.
- **Enabling Macros:** Click "Enable Content," if a security warning appears when opening the file.
- **Undo Operations:** Operations performed via VBA cannot be reversed using the standard Excel **Undo** button (Ctrl+Z). It is recommended to save the file before execution.
- **Overwriting Results:** Running various functions will sequentially overwrite the contents of the **result**, **analyze**, **better_choices**, and **identical_choices** sheets.

2 Configuration

The program has seven Excel sheets (man, group, conf, result, analyze, better_choices, and identical_choices). We display the screen-shots and explain what are shown in each sheet, before proceeding to the explanation of how to use this program.

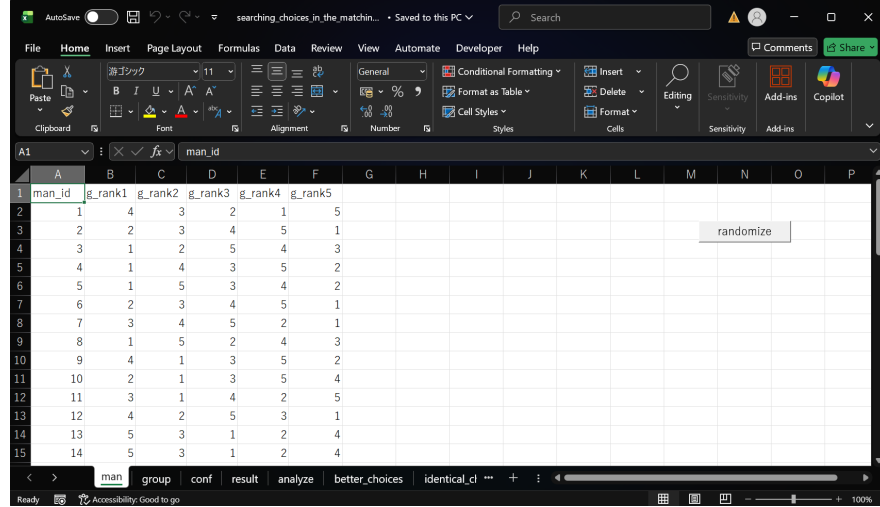
2.1 man sheet

This sheet stores the preference of each worker over divisions (Figure 2).

Column A (man_id) : Enter the ID numbers of workers.

Column B (g_rank1) and onwards : Enter the preferences of workers from left to right; In the row where man_id is i , the ID number of division j should be entered in the column labeled g_rank l , if worker i states that division j is the l -th preferred one for him or her.

randomize : Click on this button when it is needed to generate the workers' preferences randomly for an exercise.



man_id	g_rank1	g_rank2	g_rank3	g_rank4	g_rank5
1	4	3	2	1	5
2	2	3	4	5	1
3	1	2	5	4	3
4	1	4	3	5	2
5	1	5	3	4	2
6	2	3	4	5	1
7	3	4	5	2	1
8	1	5	2	4	3
9	4	1	3	5	2
10	2	1	3	5	4
11	3	1	4	2	5
12	4	2	5	3	1
13	5	3	1	2	4
14	5	3	1	2	4

Figure 2: man sheet (Workers' Preferences)

2.2 group sheet

This sheet stores the priority order of each division over workers (Figure 3).

Column A (group_id) : Enter the ID numbers of divisions.

Column B (m_rank1) and onwards : Enter the priority orders of divisions from left to right; In the row where group_id is j , the ID number of worker i should be entered in the column labeled m.rank l , if division j states that worker i is the l -th preferred one for it.

randomize : Click on this button when it is needed to generate the divisions' priority orders randomly for an exercise.

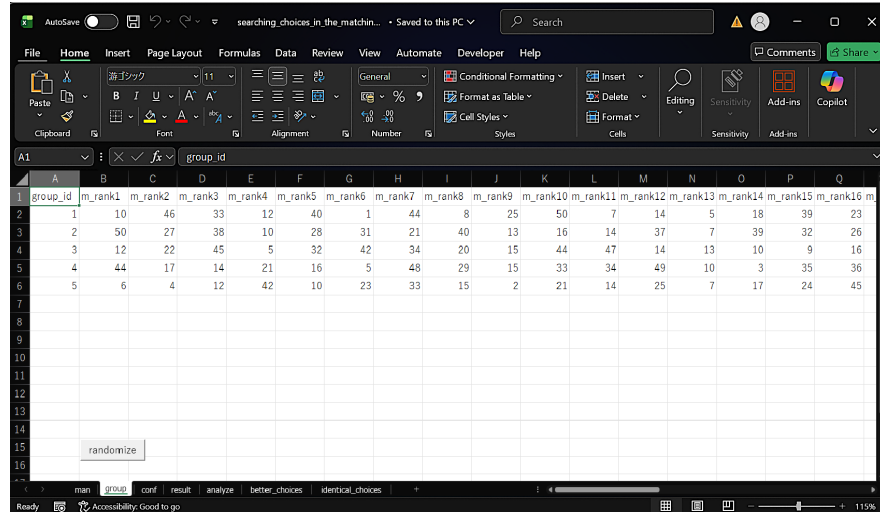


Figure 3: group sheet (Divisions' Priority Orders)

2.3 conf sheet

This sheet is used to set the operating parameters in the program (Figure 4).

Column A (group_id) : Enter the ID numbers of divisions. List all group IDs.

Column B (max) : Enter the number of capacity for each division.

Column D (num)_man : Enter the total number of workers.

Column E (num_group) : Enter the total number of divisions.

	A	B	C	D	E	F	G	H	I	J	K	L
1	group_id	max		num_man	num_group		check_other_preferences					
2	1	10		50	5		5					
3	2	10										
4	3	10										
5	4	10										
6	5	10										
7												
8												
9												
10												
11												

Figure 4: conf sheet (Configuration)

Column G (check_other_preferences) : Given **lists of preferences, priority orders, and capacities, with** an assignment of workers to divisions, this program can search **“better” choices for each worker that assign him or her to “higher-ranked” divisions** under the Boston algorithm, and it can also search **other choices for each worker that assign him or her to the same or better divisions** under the DA algorithm. See the explanations for result sheet, better_choices sheet, and identical_choices sheet for these functions in more detail. Enter the number of top k choices in workers’ preferences. When the number is set to 5, then for every worker, the program considers all possible manipulation of the top 5 divisions ranked in his or her preference when the “search Better Choices (boston)” button is pressed in the result sheet, or it permuted the top 5 divisions ranked in his or her preference to find the preferences that generate the same assignment listed in the result sheet when the “search Identical Choices (DA)” button is pressed.

2.4 result sheet

This sheet shows the matching computed by algorithms and provides buttons for many commands executed by this program (Figure 5). When we press the **da (man)** button (**da (group)**) on this sheet, the program computes the assignment using the DA algorithm in which the workers (divisions) propose to divisions (workers). Pressing **boston (man)** generates the assignment where the workers and divisions are matched by the Boston algorithm. The function of the **randomize (man)** (**randomize (group)**) is the same as the function of the **randomize** button in the man (group) sheet.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	man_id	result_group_id	last_rank(man)		group_id	last_rank(group)	max								
1	1	5	2		1	10	10								
2	2	5	2		2	10	10		da (man)		randomize man				
3	3	4	4		3	10	10								
4	4	3	2		4	10	10		da (group)		randomize group				
5	5	5	1		5	10	10								
6	6	3	1						boston (man)						
7	7	5	3												
8	8	5	2												
9	9	3	3												
10	10	4	1								eval				
11	11	4	1												
12	12	1	1								search Better Choices (boston)				
13	13	2	1												
14	14	1	1								search Identical Choices (DA)				
15	15	4	2												

Figure 5: result sheet (Matching: Results)

man_id : The ID of each worker.

result_group_id : The ID of the division to which each worker is assigned.

last_rank(man) : The rank of the division to which each worker is assigned (in the list of the Workers' Preferences; Figure 2).

group_id : The ID of each division.

last_rank(group) : The rank of the lowest-ranked worker assigned to that division (in the list of the Divisions' Priority Orders; Figure 3).

max : The capacity of each division (in the list of Configuration (Figure 4)).

The other buttons show the results in the analyze, better_choices, identical.choices sheets, respectively;

eval, **search_Better_Choices (boston)**, and **search_Identical_Choices (DA)**.

The functions are explained in the subsections corresponding to the sheets in subsections 2.5, 2.6, and 2.7, respectively.

2.5 analyze sheet

This sheet outputs the verification results of inefficient matchings by exchanging divisions to which two workers are assigned (Figure 6). When we press the **eval** button attached in the result sheet, the program examines all the exchanges of divisions for any pairs of workers. If both workers are better off and both divisions are better off by that exchange, then the matching is (Pareto) inefficient.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	manA	manB	groupA	groupB	manA_up	manB_up	groupA_up	groupB_up		man_count	group_count	double_count			
1	1	6	5	3	-1	-3	26	5		36	85	0			
2	1	14	5	1	-2	-1	16	6							
3	1	17	5	3	-1	-2	13	12							
4	1	19	5	3	-1	-3	2	6							
5	1	23	5	1	-2	-3	21	10							
6	1	25	5	1	-2	-3	15	3							
7	1	28	5	2	1	1	-11	-36							
8	1	35	5	3	-1	-3	1	4							
9	1	37	5	2	1	1	-12	-29							
10	1	38	5	2	1	1	-14	-38							
11	2	6	5	3	-2	-3	8	2							
12	2	27	5	1	1	1	-31	-23							
13	2	41	5	1	1	1	-40	-21							
14	2	50	5	1	1	1	-38	-38							

Figure 6: analyze sheet (Inefficiency Analysis: Results)

manA / manB : IDs of the workers paired for exchanging their divisions.

groupA / groupB : IDs of the original divisions to which the workers were assigned.

manA_up / manB_up : It indicates how many ranks each worker is better or worse off by the exchange. If both workers in a pair are better off, then the numbers of ranks for the improved pair is indicated in yellow.

groupA_up / groupB_up : It indicates how many ranks each division is better or worse off by the exchange. If both divisions are better off, then the numbers of ranks for the improved divisions is indicated in light blue.

man_count / group_count / double_count : It counts the number of pairs of workers who are both better off, the number of divisions which are both better off, and the number of exchanges by which both workers and both divisions are better off, respectively, by the exchange. The cells colored in light yellow (light blue) indicates the pair of manA_up and manB_up (groupA_up and groupB_up) who (which) are better off.

We can enter an assignment in the result sheet by ourselves. Given workers' preferences in the man sheet, divisions' priority orders in the group sheet, and the capacities in the conf sheet, the inefficiency of the assignment is examined by pressing the **eval** button. Without pressing the **da (man)** button, the **da (group)** button, and the **boston (man)** button. The inefficiency of the actual assignment can be tested in this way.

2.6 better_choices sheet

This sheet outputs the search results of better choices for each worker under the Boston algorithm (Figure 7). Press the button named **search_Better_Choices (boston)** in the result sheet (shown in subsection 2.4). Then, for each worker, this program searches all the possible preferences **that assign** him or her to **higher-ranked divisions**, given the preferences stated by the other workers listed in the man sheet.

man_ID	org_group	assigned_group	result_diff	g_rank1	g_rank2	g_rank3	g_rank4	g_rank5
24	1	3	1	3	5	1	2	4
24	1	3	1	3	5	1	4	2
24	1	3	1	3	5	2	1	4
24	1	3	1	3	5	2	4	1
24	1	3	1	3	5	4	2	1
24	1	3	1	3	5	4	1	2
24	1	3	1	3	1	5	2	4
24	1	3	1	3	1	5	4	2
24	1	3	1	3	1	2	5	4
24	1	3	1	3	1	2	4	5
24	1	3	1	3	1	4	2	5
24	1	3	1	3	1	4	5	2
24	1	3	1	3	2	1	5	4
24	1	3	1	3	2	1	4	5

Figure 7: better_choices sheet (Better Choices under the Boston algorithm: Results)

man_ID : The ID of the worker for whom being better choices are searched.

org_group : The ID of the division to which the worker is assigned in the result sheet.

assigned_group : The ID of the division to which the worker is re-assigned by choosing an alternative statement of his or her preference.

result_diff : The degree of improvement (rank of the division shown in assigned_group – rank of the division shown in org_group).

g_rank1, g_rank2, ... : The list of an alternative statement of the worker's preference.

2.7 identical_choices sheet

This sheet outputs the search results of preferences stated by each worker that assign him or her to the same **or higher-ranked divisions** under the DA algorithm (Figure 8). Press the button named **search_Identical_Choices** (DA) in the result sheet (shown in subsection 2.4). Then, for each worker, this program searches all the possible preferences **that assign him or her to the same division or divisions which are higher-ranked than the one** shown in the list of the result sheet, given the preferences stated by the others in the man sheet (shown in subsection 2.1).

man_ID	org_group	assigned_group	result_diff	g_rank1	g_rank2	g_rank3	g_rank4	g_rank5
1	4	4	0	4	3	2	1	5
2	1	4	4	0	4	3	2	5
3	1	4	4	0	4	3	2	5
4	1	4	4	0	4	3	1	2
5	1	4	4	0	4	3	1	5
6	1	4	4	0	4	3	5	1
7	1	4	4	0	4	3	5	2
8	1	4	4	0	4	2	3	1
9	1	4	4	0	4	2	3	5
10	1	4	4	0	4	2	1	3
11	1	4	4	0	4	2	1	5
12	1	4	4	0	4	2	5	1
13	1	4	4	0	4	2	5	3
14	1	4	4	0	4	1	2	3
15	1	4	4	0	4	1	2	5

Figure 8: identical_choices sheet (Identical Choices under the DA algorithm: Results)

man_ID : The ID of the worker for whom identical choices are searched.

org_group : The ID of the division to which the worker is assigned in the result sheet.

assigned_group : The ID of the division to which the worker is re-assigned by choosing an alternative statement of his or her preference.

result_diff : The degree of improvement (rank of the division shown in assigned_group – rank of the division shown in org_group) which should be zero if the search results is correct.

g_rank1, g_rank2, ... : The list of an alternative atatement of the worker's preference.

3 Operation

3.1 Executing Matching Algorithms: Basic Function

Execute the algorithms using the following buttons on the result sheet (Figure 5):

- **da (man)**: Executes the worker-proposing DA algorithm.
- **da (group)**: Executes the division-proposing DA algorithm.
- **boston (man)**: Executes the worker-proposing Boston algorithm.

The assignment of worker is shown in the left side of the result sheet. Read subsection 2.4 for the instructions on how to interpret the result.

In variants of the Boston algorithm, every division that reaches its full capacity in a certain round is removed from the list of candidates to which workers can be assigned in the next round. This treatment implies that some workers skip the review of their l -th preferred divisions and proceed to the next round to be reviewed for their $l + 1$ -st preferred divisions. In this software for the classical Boston algorithm, no division that reaches its capacity is not removed from the candidates for assignment and workers' preferences, and the rounds in which workers are reviewed are aligned for all workers who are eligible for the review.

3.2 Welfare Evaluation: Inefficient Matching

Click on the **eval** button attached in the result sheet. Then, the program examines all the exchanges of divisions for any pairs of workers. If both workers are better off and both divisions are better off by that exchange, then the matching is (Pareto) inefficient. The examination result is shown in the **analyze** sheet. Read subsection 2.5 for the instructions on how to interpret the result. Figure 6 shows an example of the result. The color coding implies the following meaning:

- **Light Yellow**: Indicates the improvement in ranking of the division in the worker's preference.
- **Light Blue**: Indicates the improvement in ranking of the worker in the group's priority order.
- **Light Green**: Indicates that both of the above have improved, meaning the matching is Pareto inefficient.

Pareto inefficiency is not defined for a reassignment which is generated by swapping of two workers between their divisions in an assignment. Simply, the improvement indicated in light green implies that the assignment is Pareto inefficient. A matching μ is **Pareto efficient**, if there is no such a reassignment of workers that improves the welfare of some workers and divisions in terms of workers' individual rankings of preferences and divisions' priority orders without making the welfare of any workers and divisions worse.

We can also test the inefficiency of existing assignments by entering some assignment directly into the **result** sheet and running **eval** without clicking the algorithm buttons.

3.3 Preference Manipulation: Strategic Aspects

Each worker can state any preference including his or her true preference, when he or she is asked to state **some preference**. Accordingly, this situation involves conflicts of interest with the decisions of other workers and divisions, resulting in a strategic “game” for each worker regarding what preferences to state and for each division regarding what priority order to report. In this program, two aspects of this situation are treated with divisions' priority orders kept intact.

- **search Better Choices (boston)**: Searches other statements of the preference of each worker that generate **more favorable assignments** for **him or her** under the Boston algorithm, with the preferences stated by the **others** kept intact, and outputs the search results for all the workers to the **better_choices** sheet. Read subsection 2.6 for more details and see Figure 5 and Figure 7.
- **search Identical Choices (DA)**: Verifies if preference manipulation by each worker can **assign him or her to the same or higher-ranked divisions** under the worker-proposing DA mechanism, with preferences stated by the **others** kept intact, and outputs the results to the **identical_choices** sheet. Read subsection 2.7 for more details and see Figure 5 and Figure 8.

4 Troubleshooting

4.1 Developer Ribbon is Not Displayed

If the "Developer" ribbon indicated by a light green ruled line in Figure 9 does not appear in Excel, configure it using the following steps:

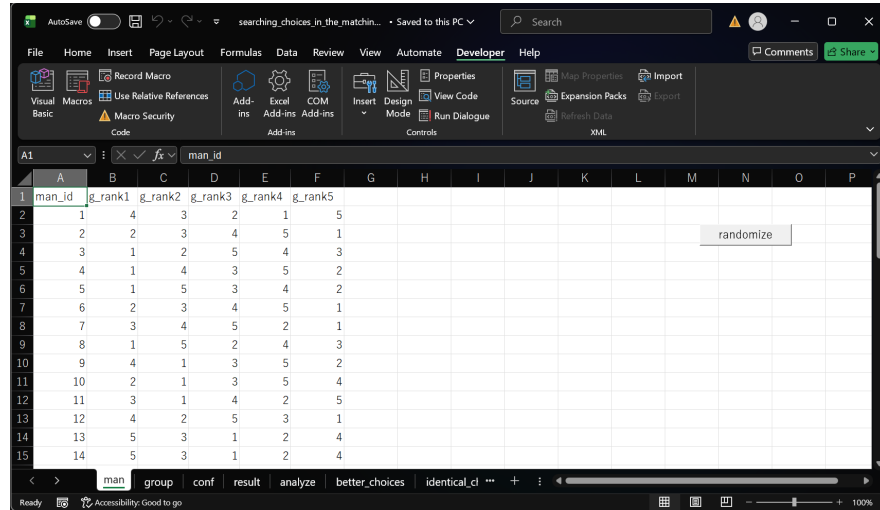


Figure 9: Developer ribbon

1. Click on the **File** tab.
2. Select **Options** → **Customize the Ribbon** in **Excel Options** window (Figure 10).
3. Check the check box for **Developer** in the "Main Tabs" on the right-hand side of the window.
4. Click on the OK button at the bottom-right edge of the window.
5. Close the Excel Options window.

4.2 Error Message is Displayed

Check the following things if a runtime error occurs.

- Do the numbers of workers and groups set in the conf sheet (Figure 4) match the actual number put in the man sheet and the group sheet?

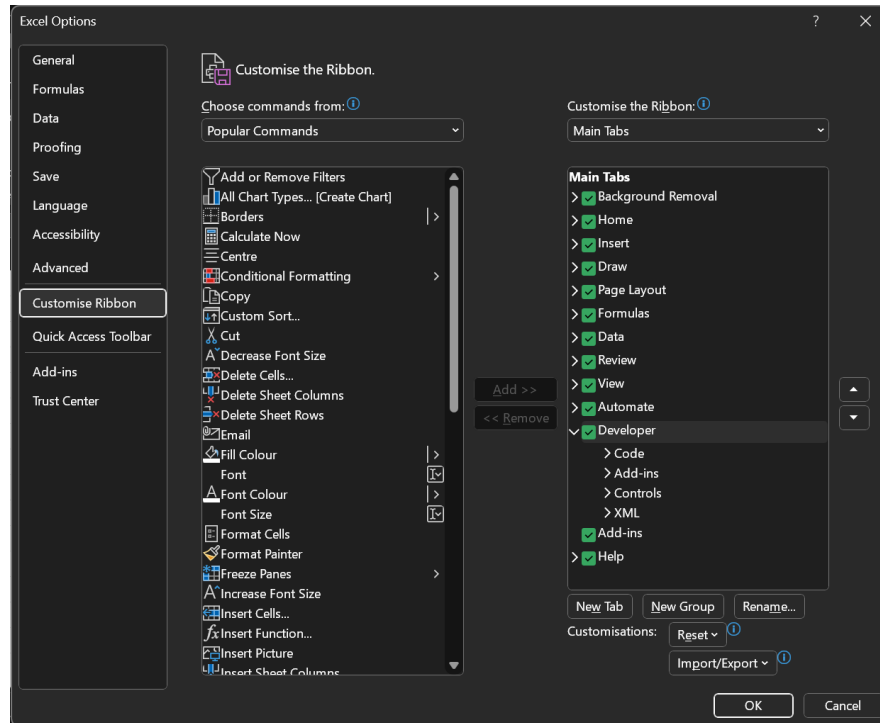


Figure 10: Excel Options window

- Are all input values (ID, rank, capacity) entered as single-byte characters in the man sheet, the group sheet, and the conf sheet?
- Is the total number of workers the same as the total sum of all capacities in the conf sheet?

4.3 Results are Incorrect (without Error Message)

- Ensure whether Excel file is not open in “Protected View.” If it is, click on **Enable Editing** button to end Protected View.
- Check for duplicate numbers or missing ranks in the workers’ preferences and the division’s priority orders.
- There should be some cases where the calculation may take time a long time in the analyses of preference manipulation. In such cases, reset the number of top rankings of workers’ preferences that are examined for changing preferences stated by them at `check_other_preferences` in the conf sheet. (See Figure 4.)

Caveat: This program will display a run-time error message and basically generate no assignment, if there are missing values in preferences or priority orders. (A matching may be generated even if some values are missing in divisions' priority orders of workers when the worker-proposing DA algorithm is used. Make complete lists of rankings (preferences and priority orders in using this program.

The codes of this program were not written for the consecutive execution. If you use this program in such a way, then the codes may refer to numerical values remaining in the cache on your PCs and generate incorrect matchings. In order to surely avoid this event, clear cache or close your Excel file for every execution.